

Jeu de Tapa

Auteurs: ALIAN Nastaran, DUPRAZ Cha, GHAREHBAGHI Donya et SKUBISZ Magdalena

Introduction

Nous avons choisi le jeu de Tapa d'une part pour sa familiarité, plusieurs membres du groupes connaissant déjà ce jeu, et d'une autre part pour sa complexité. 3 règles principales apparaissent, sur la topologie local, sur la topologie globale des "rivières" (zones noir), mais aussi sur des contraintes d'arrangement (les indices).

1. Règles du Jeu de Tapa

1. **Couleur des cases** : Chaque case est soit noire, soit blanche
2. **Pas de carrés noirs 2×2** : Aucun groupe de 4 cases noires adjacentes ne peut former un carré
3. **Numéros sur les cases** : Les indices donnent la longueur des blocs noirs consécutifs dans les cases adjacentes. Le nombre de numero indique le nombre de voisinages distincts.
4. **Continuité des cases noires** : Toutes les cases noires doivent former une région connexe orthogonalement (pas de cases noir isolées)

Les règles du jeu Tapa peuvent être ramenées à trois contraintes fondamentales : **pas de carrés noirs 2×2**, **respect des indices numériques** et **continuité des cases noires**. Ces trois contraintes forment l'**ensemble minimal et complet** caractérisant une grille Tapa valide.

1.1. Une caractérisation minimale

Chacune contraintes capture uneregle distincte et indépendante de la solution :

- **Carrés 2×2** : contrainte **locale et négative** — interdit certaines configurations de 4 cases adjacentes. Elle agit sur la *forme* des régions noires.
- **Indices numériques** : contrainte **locale et positive** — fixe les positions possible du voisinage d'une case indiquée. Elle agit sur le *nombre* et la *taille* et la *position* des blocs noirs autour des indices.
- **Continuité** : contrainte **globale** — impose une propriété structurelle sur l'ensemble du graphe des cases noires. Elle agit sur la *cohésion* de la solution.

Contrainte	Échelle	Type de modélisation
Carrés 2×2	Locale (4 cases)	Clauses négatives directes en FNC
Indices	Voisinage (jusqu'à 8 cases)	FND (arrangements) → FNC
Continuité	Globale (toute la grille)	Variables auxiliaires de distance → FNC

Ces trois contraintes sont **indépendantes** : aucune ne peut se déduire des deux autres.

Toute solution satisfaisant simultanément ces trois contraintes est une solution valide du puzzle, et réciproquement.

1.2. Passage en format dimac:

Les trois règles étant indépendantes, nous pouvons initialiser un fichier dimacs et les ajouter successivement sans ordre à celui-ci. Il est quand même nécessaire de garder en mémoire le nombre de clauses ajoutées et le nombre de variables supplémentaire pour mettre à jour l'entête de ce fichier à la fin de l'exécution.

Modélisation des règles

2. Indexation des Cases

Nous représentons la grille comme un tableau à deux dimensions de taille `nombre_lignes × nombre_colonnes` indexé à 1.

- Les indices de cases vont de 1 à `nombre des cases`
- **Formule d'indexation** : $\text{indice_case} = 1 + \text{numéro_colonne} + \text{numéro_ligne} \times \text{nombre_colonnes}$

L'index d'une case représente son nom pour le format DIMACS.

Exemple :

```
Position (ligne=1, colonne=2) dans une grille 3×3
→ Index = 1 + 2 + 1 × 3 = 6
```

3. Couleur des cases

Variable X pour déterminer la couleur d'une case :

```
X ∈ {
    0 → case blanche
    1 → case noire
}
```

4. CONTRAINTE : pas de carrés noirs 2×2

Configuration Valide		Configuration Invalide
<pre> ----- ----- ----- ----- \$\$\$ \$\$\$ \$\$\$ \$\$\$ \$\$\$ \$\$\$ ----- ----- ----- ----- \$\$\$ \$\$\$ ----- ----- ----- ----- \$\$\$ \$\$\$ \$\$\$ \$\$\$ ----- ----- ----- ----- </pre>	---	<pre> ----- ----- ----- ----- \$\$\$ \$\$\$ \$\$\$ \$\$\$ ----- ----- ----- ----- \$\$\$ \$\$\$ \$\$\$ \$\$\$ \$\$\$ \$\$\$ ----- ----- ----- ----- ----- ----- ----- ----- </pre>

Règle : Aucun groupe de 4 cases noires adjacentes ne peut former un carré

Modélisation en logique: pour tout ensemble des cases $\{i, i+1, i + \text{nombre_lignes}, i + \text{nombre_lignes} + 1\}$ note $\{a,b,c,d\}$ on a:

$-a + -b + -c + -d$

"au moins une de ces 4 case n'est pas noir"

Note : Lors du parcours de la grille pour generer les caluses si une des quatre cases est forcée blanche par un nombre (regle 3), cet ensemble pourrait etre ommit

5. CONTRAINTE: Numeros sur les cases

Les indices (0) donnent la longueur des blocs noirs consécutifs (1) dans les cases adjacentes (2).
Le nombre de numéros (4) indique le nombre de voisinages (5) distincts.

Grille en entré		Grille résolue
----- ----- ----- -----		----- ----- ----- -----
1,1		1,1 \$\$\$ \$\$\$
		\$\$\$ \$\$\$
----- ----- ----- -----		----- ----- ----- -----
7 3		\$\$\$ 7 \$\$\$ 3
		\$\$\$ \$\$\$
----- ----- ----- -----	---->	----- ----- ----- -----
		\$\$\$ \$\$\$ \$\$\$
		\$\$\$ \$\$\$ \$\$\$
----- ----- ----- -----		----- ----- ----- -----
2		\$\$\$ 2
		\$\$\$
----- ----- ----- -----		----- ----- ----- -----

5.1. Formalisation:

La case d'index p d'un indice (0) est toujours blanche.

Un indice (0) est une case de la grille avec au moins un numéro inscrit dedans. Toutes les restrictions imposées par un indice (0) sont respectées.

Un voisinage (5) est un ensemble de cases noires consécutives (1) sur le graphe C . Deux voisinages distincts sont séparés par au moins une case blanche.

L'indice (0) contient un ensemble de nombres E composant les restrictions imposées par l'indice. La taille de l'ensemble E correspond au nombre de voisinages (5) distincts adjacents à l'indice. La valeur des éléments (4) de E correspond à la taille de ces voisinages. Les voisinages n'ont pas d'ordre.

Puisque deux voisinages (5) distincts sont séparés par au moins une case blanche et qu'il n'y a pas de restrictions directes sur le nombre de cases blanches, un voisinage de taille n peut être orienté et représenté sur $n + 1$ cases, n cases noires consécutives (1) suivies par une case blanche. Cette représentation des voisinages sera celle utilisée par la suite pour imposer la distinction.

Pour un graphe cyclique C de l cases consécutives (1) et un ensemble F , imposé par un indice (0) à l'index p , de voisinages (5) distincts et cumulativement représenté sur l cases ou moins, chaque arrangement $A_i \in A$ de ces voisinages sur C est modélisé par une conjonction λ_i de littéraux, chaque littéral représentant l'état de la case pour cet arrangement (par défaut blanc si non restreint).

Il existe un arrangement $A_i \in A$ respecté pour que l'indice (0) à l'index p soit respecté.

5.2. Modélisation:

Les restrictions d'un indice (0) à l'index p sont représentées pour $l = 8$, sur le graphe cyclique défini tel que $C = (V, E)$ avec $V = \{a,b,c,d,e,f,g,h\}$, $E = \{(a,b),(b,c),(c,d),(d,e),(e,f),(f,g),(g,h),(h,a)\}$ sur l'ensemble $\{p, p-\text{nombre_colonnes}-1, p-\text{nombre_colonnes}, p-\text{nombre_colonnes}+1, p+1, p+\text{nombre_colonnes}+1, p+\text{nombre_colonnes}, p+\text{nombre_colonnes}-1, p-1\}$ noté $I = \{x,a,b,c,d,e,f,g,h\}$, C correspondant aux cases adjacentes (2) à $\{x\}$ ($\forall v \in V, v \sim x$)

5.3. Distinctions de cas

5.3.0.1. Indice libre:

Un indice (0) est dit libre si toutes les cases adjacentes (V) sont dans la grille (l'indice n'est pas sur un bord/angle) et si aucune des cases adjacentes n'est aussi un indice (dans ce cas : $R = \emptyset$). Dans ce cas tous les arrangements $A_i \in A$ sont possibles, et nous considérerons la FND induite de la disjonction des monomes λ_i

5.3.0.1. Indice restreint:

Un indice (0) est dit restreint si au moins une case adjacente (V) n'est pas dans la grille (l'indice est sur un bord/angle) ou si au moins une des cases adjacentes est aussi un indice. L'ensemble des cases restrictives de l'indice est dénoté $R \subset V$. Dans ce cas seuls les arrangements $A' \subset A$ où aucun des éléments de R n'est noir dans les arrangements A' . Nous considérerons la FND induite de la disjonction des monomes λ_k de chaque arrangement $A'_k \in A'$

En pratique, lors de l'évaluation des arrangements, seulement les arrangements A' sont calculés par le programme

5.4. Passage de la FND à la FNC:

La transformation de la FND vers la FNC est effectuée de manière incrémentale, par distributivité explicite.

La FNC est initialisée à partir du premier monôme, en convertissant ses littéraux en clauses unitaires.

Ensuite, pour chaque monôme suivant de la FND, les littéraux sont intégrés dans la FNC par mise à jour des clauses existantes:

- chaque clause de la FNC est combinée avec chaque littéral du monôme courant
- les littéraux redondants sont supprimés au sein des clauses
- les clauses valides sont éliminées
- les doublons de clauses sont supprimés après chaque itération.

Ce processus est répété séquentiellement pour tous les monômes de la FND, produisant une FNC équivalente sans génération explicite de la forme développée par distributivité.

5.4.1. Optimisation pour le cas des indices libres

Dans le cas d'un indice libre, il peut être noté que tous les indices de même forme ont la même FND/FNC à morphisme près. La FNC de la majorité des indices libres étant de l'ordre des 5000 clauses, pour éviter de refaire la distribution qui est computationnellement importante, nous pouvons pré-calculer et transcrire les FNC dans des fichiers (présents dans l'archive sous : "/indices/indice_abcd.clauses") avec la fonction *initialise_clauses()* dans indices.c et les récupérer avec *lire_clause(a,b,c,d,<Liste de destination>)*. [Cela permet de transcrire les indices d'une grille de 10x10 en ~0.1s contre ~1s sans cette optimisation]

5.5. Passage en format dimac:

Chaque clause de la FNC étant représentée sur un tableau à 8 colonnes, les colonnes représentent respectivement a, b, c, d, e, f, g, h tel que défini sur *I*. Les valeurs possibles de chaque élément sont :

```
-1 : littéral non présent dans la clause
0  : négation du littéral présente dans la clause
1  : littéral présent dans la clause
```

Note : par construction, toutes les valeurs représentées par les littéraux de la FNC et de p sont dans $I \setminus R$ et sont donc définies sur la grille.

Pour chaque indice nous pouvons inscrire dans le fichier le commentaire suivant :

```
c Clauses pour la case (i,j) avec indice k0 ... k3
```

Où i est l'indice de colonne et j l'indice de ligne induit de p , et $k_0 \dots k_3$ les tailles des voisinages de l'indice (0). Pour n voisinages, seulement de k_0 à $k_{(n-1)}$ sont inscrits

Suivi de la clause unitaire représentant la case de l'indice qui doit être blanche

```
p 0
```

Chaque clause de la FNC est ensuite inscrite dans le fichier.

Exemple (Indice 5 sur un bord de grille):

```
c Clauses pour la case (5,6) avec indice 5
-55 0
54 0
64 0
65 0
66 0
44 0
```

Note 1 : En utilisant la propriété suivante : deux arrangements A_j, A_k sont équivalents si $\lambda_j \equiv \lambda_k$ lors du calcul des arrangements nous pouvons obtenir un ensemble A distinct (qui évitera les doublons par la suite)

Note 2: l'ensemble des clauses de la FNC et des monômes de la FNC est représenté dans le programme par une liste chaînée de tableaux, ce qui nous permet de faire une insertion sans doublon efficace.

Note 3 : L'indice choisi est volontairement très court, la taille des FNC étant majoritairement soit inférieure à 8 clauses, soit (rarement) entre 50 et 80 clauses, soit de l'ordre des 5000 clauses.

Note 4 : Le nombre de clauses ajoutées par un indice (plus particulièrement leur somme pour tous les indices) est remonté à la fin de l'inscription des clauses dans le fichier, pour mettre à jour l'entête du fichier .dimacs.

6. CONTRAINTE: Continuité des cases noires



6.1. Formalisation

On traite la grille comme un graphe $A = (V_A, E_A)$ où chaque case représente un sommet, et deux sommets sont reliés par une arête s'ils sont orthogonalement adjacents (chaque sommet a donc 2, 3 ou 4 arêtes selon sa position dans la grille).

On définit également le sous-graphe induit B de A par l'ensemble des cases noires. La règle de continuité des cases noires est respectée si et seulement si B est connexe.

Note : Conformément à notre version des règles du Tapa, la relation d'adjacence est strictement orthogonale (haut, bas, gauche, droite). Les diagonales ne comptent pas comme connexion.

6.2. Modélisation

Vérifier directement la connexité d'un graphe par une formule CNF n'est pas trivial, car la connexité n'est pas une propriété locale. Nous utilisons donc un encodage par **arbre couvrant orienté** (spanning tree encoding), basé sur une fonction de distance à une racine.

6.2.1. Principe

Le sous-graphe B est connexe si et seulement s'il existe un arbre couvrant sur B . Nous forçons l'existence d'un tel arbre en assignant à chaque case noire une **distance** par rapport à une case racine unique :

- La racine a distance 0.
- Toute autre case noire a une distance $d \geq 1$ et possède au moins un voisin orthogonal noir de distance $d - 1$.

Par induction sur d , chaque case noire est reliée à la racine par une chaîne de voisins orthogonaux noirs, ce qui garantit la connexité de B .

6.3. Variables auxiliaires

On introduit pour chaque case d'index $v \in \{1, \dots, \text{nombre_cases}\}$ et chaque distance $d \in \{0, 1, \dots, D_{\max}\}$ une variable booléenne $D_{v,d}$:

```
D_{v,d} ∈ {
  0 → la case v n'est pas à distance d de la racine
  1 → la case v est à distance d de la racine
}
```

où $D_{\max} = \text{nombre_cases} - 1$ est la distance maximale possible dans la grille.

Au format DIMACS, ces variables sont nommées à la suite des variables de couleur :

```
nombre_cases + (v-1) × nombre_cases + d + 1 .
```

6.4. Contraintes pour la continuité

6.4.1. (1) Lien entre couleur et distance

Une case a une distance si et seulement si elle est noire.

(1a) Si une case v est noire, alors elle a au moins une distance assignée :

$$X_v \implies \bigvee_{d=0}^{D_{\max}} D_{v,d}$$

En CNF, pour chaque case v :

```
-X_v + D_{v,0} + D_{v,1} + ... + D_{v,D_max}
```

(1b) Si une case v a une distance d assignée, alors elle est noire :

$$D_{v,d} \implies X_v$$

En CNF, pour chaque case v et chaque distance d :

$$-D_{\{v,d\}} + X_v$$

6.4.2. (2) Unicité de la distance

Chaque case noire a **exactement une** distance. La disjonction (au moins une) est déjà donnée par (1a). Il reste à imposer **au plus une**, encodé par paires :

Pour chaque case v et chaque paire (d_1, d_2) avec $d_1 < d_2$:

$$\neg(D_{v,d_1} \wedge D_{v,d_2})$$

En CNF :

$$-D_{\{v,d_1\}} + -D_{\{v,d_2\}}$$

6.4.3. (3) Unicité de la racine

Pour que l'arbre couvrant soit unique, il ne peut exister qu'une seule racine sur toute la grille. Si deux cases étaient racines (distance 0), on aurait deux arbres disjoints donc deux composantes connexes.

Pour toute paire de cases (u, v) avec $u < v$:

$$\neg(D_{u,0} \wedge D_{v,0})$$

En CNF :

$$-D_{\{u,0\}} + -D_{\{v,0\}}$$

6.4.4. (4) Existence conditionnelle de la racine

Si au moins une case est noire, alors une racine doit exister. Si toutes les cases sont blanches, aucune racine n'est nécessaire.

Pour chaque case v :

$$X_v \implies \bigvee_{u=1}^{\text{nombre_cases}} D_{u,0}$$

En CNF :

$$-X_v + D_{\{1,0\}} + D_{\{2,0\}} + \dots + D_{\{\text{nombre_cases},0\}}$$

6.4.5. (5) Propagation de la distance

C'est la contrainte centrale de l'encodage. Elle force chaque case noire non-racine à être adjacente orthogonalement à une case de distance immédiatement inférieure, ce qui garantit l'existence d'une chaîne vers la racine.

Soit $N_{\perp}(v)$ l'ensemble des voisins **orthogonaux** de v dans la grille (au plus 4, moins aux bords et aux coins).

Pour chaque case v et chaque distance $d \in \{1, 2, \dots, D_{\max}\}$:

$$D_{v,d} \implies \bigvee_{u \in N_{\perp}(v)} D_{u,d-1}$$

En CNF :

$$-D_{\{v,d\}} + D_{\{u_1,d-1\}} + D_{\{u_2,d-1\}} + \dots + D_{\{u_k,d-1\}}$$

où $u_1, \dots, u_k$ sont les voisins orthogonaux de v présents dans la grille.

6.4.6. Articulation des contraintes

Les contraintes (3), (4) et (5) travaillent conjointement :

- (3) garantit l'**unicité** de la racine (sinon, plusieurs arbres disjoints).
- (4) garantit l'**existence** de la racine (sinon, pas d'arbre possible pour les cases noires).
- (5) garantit que chaque case noire est **connectée** à la racine par une chaîne orthogonale.

Les contraintes (1) et (2) assurent la cohérence entre la coloration et les distances assignées.

6.4.7. Passage en format DIMACS

Une fois la modélisation logique de la contrainte de continuité établie (variables auxiliaires $D_{v,d}$ et les cinq familles de contraintes (1a), (1b), (2), (3), (4) et (5)), il reste à traduire cette formule logique en un fichier au format DIMACS exploitable par le solveur SAT.

6.4.7.1. La formule est déjà en FNC

Contrairement à la contrainte des indices numériques, qui nécessite un passage de la Forme Normale Disjonctive (FND) vers la FNC, **la contrainte de continuité est directement formulée en FNC** dès l'étape de modélisation. En effet, chacune des cinq familles de contraintes produit naturellement des **clauses** (disjonctions de littéraux) :

Contrainte	Forme logique	Type de clause générée
(1a)	$\neg X_v \vee \bigvee_d D_{v,d}$	Clause longue de taille $D_{\max} + 2$
(1b)	$\neg D_{v,d} \vee X_v$	Clause binaire
(2)	$\neg D_{v,d_1} \vee \neg D_{v,d_2}$	Clause binaire
(3)	$\neg D_{u,0} \vee \neg D_{v,0}$	Clause binaire
(4)	$\neg X_v \vee \bigvee_u D_{u,0}$	Clause longue de taille $N + 1$
(5)	$\neg D_{v,d} \vee \bigvee_{u \in N_{\perp}(v)} D_{u,d-1}$	Clause de taille 3 à 5

L'ensemble de ces clauses, reliées par conjonction, forme bien une formule en FNC. Aucune transformation supplémentaire n'est nécessaire avant l'écriture dans le fichier DIMACS.

6.4.7.2. Numérotation des variables

Le format DIMACS exige que les variables soient identifiées par des **entiers strictement positifs**. Une variable est représentée par son entier positif, sa négation par l'entier négatif correspondant. Notre projet utilise deux familles de variables :

6.4.7.2.1. Variables de couleur X_v

Ces variables existent déjà à l'étape précédente du projet (contrainte des carrés 2×2 , indices). Elles occupent les identifiants $\{1, 2, \dots, N\}$ où $N = \text{nombre_lignes} \times \text{nombre_colonnes}$. La formule d'indexation est :

$$\text{indice_X} = 1 + \text{numéro_colonne} + \text{numéro_ligne} \times \text{nombre_colonnes}$$

6.4.7.2.2. Variables auxiliaires de distance $D_{v,d}$

Pour la contrainte de continuité, nous introduisons $N \times N$ variables auxiliaires $D_{v,d}$, une pour chaque paire (case, distance possible). Pour éviter toute collision avec les identifiants de X_v , ces variables sont numérotées à la suite des variables de couleur, à partir de $N + 1$.

La formule d'indexation est :

$$\text{indice_D}(v, d) = N + (v - 1) \times N + d + 1$$

avec $v \in \{1, \dots, N\}$ et $d \in \{0, \dots, D_{\max}\}$.

Ces variables occupent donc l'intervalle $[N + 1, N + N^2]$.

Exemple pour une grille 3×3 ($N = 9$) :

Variable	Identifiant DIMACS
$X_1, X_2, \dots, X_9$	$1, 2, \dots, 9$
$D_{1,0}$	$9 + (1 - 1) \times 9 + 0 + 1 = 10$
$D_{1,1}$	$9 + 0 + 1 + 1 = 11$
$D_{1,8}$	$9 + 0 + 8 + 1 = 18$
$D_{2,0}$	$9 + 9 + 0 + 1 = 19$
$D_{9,8}$	$9 + 72 + 8 + 1 = 90$

L'identifiant maximal est donc $N + N^2 = 9 + 81 = 90$ pour cet exemple.

6.4.7.3. Écriture des clauses

Chaque clause est écrite sur une ligne du fichier, sous la forme d'une suite d'entiers (séparés par des espaces) terminée par `0`. Les littéraux positifs (variable affirmée) sont notés directement, les littéraux négatifs (variable niée) sont préfixés par `-`.

Exemple pour la contrainte (1a) appliquée à la case $v = 1$ d'une grille 3×3 :

$$\neg X_1 \vee D_{1,0} \vee D_{1,1} \vee D_{1,2} \vee \dots \vee D_{1,8}$$

s'écrit dans le fichier DIMACS :

```
-1 10 11 12 13 14 15 16 17 18 0
```

Exemple pour la contrainte (5) appliquée à la case centrale $v = 5$ à la distance $d = 2$ (ses voisins orthogonaux sont $v = 2, 4, 6, 8$) :

$$\neg D_{5,2} \vee D_{2,1} \vee D_{4,1} \vee D_{6,1} \vee D_{8,1}$$

s'écrit :

-48 20 38 56 74 0

Note: les identifiants étant calculés selon la formule ad hoc

Jeu de tests

La construction des règles nous permet de tester chaque règle individuellement ou en conjonction. Nous avons créé un jeu de tests fonctionnel pour vérifier le bon fonctionnement des règles, et vérifier qu'elles fonctionnent correctement quand elles sont appliquées ensemble. Ces tests vérifient à la fois des cas satisfaisables et des cas insatisfaisables.

Le fichier de test est construit ainsi:

```
/JeuTests/OK/indices
    /indices-continuite
    /continuite
    /2x2
    /2x2-indices
    /2x2-continuite
    /2x2-indices-continuite
/ERREUR <même sous fichiers>
```

Les tests présents ont été spécifiquement construits pour chaque différentes combinaisons de règles (les illustrations ad hoc font partie de ces tests). Pour pouvoir tester les combinaisons de règles qui n'ont pas la contrainte « indice », nous avons changé le format de grille pour accepter d'avoir des cases précises forcées blanches ou noires.

À ce jeu de tests construit manuellement, nous avons ajouté dans tous les sous-dossiers de OK au moins 4 grilles importées, différentes pour chaque dossier.

Un fichier `/GrillesImporte` contient un grand nombre de grilles importées de www.puzzle-tapa.com. La construction de ces grilles étant correcte, chacune de ces grilles peut être lancée avec toute combinaison de règles et est satisfaisable.

Nous n'avons pas créé de test de robustesse sur le format des grilles, car peu cohérent : les grilles ayant des contraintes de construction, nous avons assumé que le format d'entrée a été respecté.

Note 1 : seules 3 grilles de 15x15 ont été utilisées dans le dossier de test pour les combinaisons de règles incluant la continuité pour des raisons de temps.

Note 2 : certains tests sont insatisfaisables pour leurs règles mais satisfaisables pour un sous-ensemble de règles ; réciproquement, certains tests dans OK sont satisfaisables pour les règles données, mais insatisfaisables dès lors qu'on ajoute une règle supplémentaire.

Conclusion

Le passage de l'ensemble des tests formulés, ainsi que de toutes les grilles issues du site www.puzzle-tapa.www que nous avons importées, s'étant révélé satisfaisant et correctement exécuté, et compte tenu de la formalisation de chacune des règles ainsi que du respect rigoureux de cette formalisation dans leur retranscription en C, nous avons un haut niveau de confiance dans le fait que notre programme soit capable de résoudre toute grille de Tapa, quelle qu'en soit la taille, sous réserve de disposer de suffisamment de temps et de mémoire.

6.5. Limitations:

Ce programme prend en entrée des grilles Tapa, celles-ci peuvent être satisfaisables ou insatisfaisables, mais il est attendu qu'elles respectent un format attendu. Les indices illégaux provoquent un comportement indéfini.

Les grilles peuvent être rectangulaires, les grilles d'autres formes ne sont pas supportées.

Le temps d'exécution de la contrainte de continuité est exponentiel : résoudre une grille de 15x15 prend ~15 s et une grille de 20x20 ~1 min 20 s. Le temps et le nombre de variables rendent la résolution de grilles plus grandes fonctionnellement difficile.

Les 2 autres règles s'exécutent en $O(N)$, N : nombre de cases ou nombre d'indices respectivement.

Pré-requis:

- compilateur C installé sur sa machine @ (eg: gcc, clang...) le Makefile prend le compilateur par défaut de la machine
- Makefile installé sur sa machine @ (apt install make / apt install build-essential)

Le SAT-solver utilisé est picosat @ <https://fmv.jku.at/picosat/>

Les modules picosat.c et picosat.h ont été copiés depuis l'archive <https://fmv.jku.at/picosat/picosat-965.tar.gz>

Aucune installation du SAT solver n'est requise, les fichiers sont déjà présents dans l'archive fournie.

COMPILATION : Pour compiler ou recompiler, copiez la commande suivante dans votre terminal à l'emplacement où l'archive a été extraite : make

Pour nettoyer les fichiers créés lors de la compilation : make clean

Sources extérieures : Les grilles nommées <taille>x<taille>Normal<identifiant> et <taille>x<taille>Hard<identifiant> ont été extraites de <https://www.puzzle-tapa.com/print.php>
SAT-solver : <https://fmv.jku.at/picosat/>

Utilisation

```
./TapaSolver <flags> <fichier>
```

```
| Flag: | Règle activée |  
| 2 | Carrés 2x2 |  
| i | Indices/clues |  
| c | Continuité des cases noires |
```

Les flags peuvent être combinés. Par exemple, pour activer toutes les règles :
./TapaSolver 2ic tests/ma_grille